

Lab exercises

Part 09 —  Model Context Protocol · 09-mcp/lab/EXERCISES.md

Lab — Model Context Protocol

Work top to bottom. Each exercise has a **stated deliverable**. Everything runs offline against `../code/minimal_mcp_server.py` and `../code/mcp_client_demo.py` with the repo venv:

```
/Users/srinip/ai-all/.venv/bin/python 09-mcp/code/mcp_client_demo.py
```

No API keys, no network access, no external MCP servers required.

1. Add a fourth tool, on purpose making a naming mistake first (1h)

Open `../code/minimal_mcp_server.py`.

- **1a.** Add a new tool called `handle_it` that adds a book to `CATALOG` given a title, author, genre, and year. Give it the docstring `"""Handles the thing."""`. Register it with `@mcp.tool()`.
- **1b.** Run `mcp_client_demo.py`'s `tools/list` step (add a call if needed) and look at what a client actually receives for this tool. Write two sentences on why a model deciding whether to call `handle_it` for "add Foundation by Asimov to the catalog" is worse off than if the tool were named and described properly — be specific about what information is missing, not just "it's vague."
- **1c.** Rename it to `add_book` and write a real docstring following the pattern of `search_catalog`'s (state the exact semantics: what happens if the title already exists?). Add that missing-title-collision case as an explicit `ToolError`, not silent overwrite.

Deliverable: the final `add_book` tool code, plus the 1b paragraph, plus one sentence naming which section of `notes/01` (§6) this exercise is a worked example of.

2. Break the error-handling contract, then fix it (45 min)

- **2a.** In `search_catalog`, temporarily replace the `raise ToolError(...)` line with `return f"No catalog entries match {query!r}."` (an ordinary `return`, not a `raise`). Run `mcp_client_demo.py`'s "no match" call again and inspect the printed `tools/call result` — specifically the `is_error` field.
- **2b.** Explain, in your own words, why a model or a client UI reading this result would now treat a *failed* search as a *successful* one, and describe one concrete downstream consequence (e.g. what would an agent built on this tool tell the user?).
- **2c.** Revert the change. Now do the same experiment on `get_book`: what happens if you catch `ResourceNotFoundError` internally and `return` an error string from `get_book` instead of raising? Is the

failure mode the same as 2a, worse, or different? (Hint: resources have no `is_error` field at all — reread `notes/01` §6's resource paragraph.)

Deliverable: answers to 2b and 2c (3-5 sentences each), citing the specific JSON field(s) that changed.

3. Diagnose a primitive-selection bug (45 min)

You're handed this (broken-by-design) server sketch:

```
@mcp.tool()
def get_current_user_profile() -> str:
    """Fetch the current user's profile: name, email, and account tier."""
    return fetch_profile_from_db(current_user_id())

@mcp.resource("actions://send_invoice/{customer_id}")
def send_invoice(customer_id: str) -> str:
    """Generate and email an invoice to the given customer."""
    invoice = generate_invoice(customer_id)
    email_customer(customer_id, invoice)
    return f"Invoice sent to {customer_id}"
```

- **3a.** Both primitive choices here are backwards relative to `notes/01` §2's control-locus table. Say which primitive each function *should* be, and justify each using the "who decides when this runs" test.
- **3b.** For `send_invoice` specifically, describe the concrete failure scenario this causes: what could a host application legitimately do with a "resource" that has this side effect, and why is that dangerous here specifically (tie it to `notes/01` §2's resource-contract violation paragraph)?
- **3c.** Rewrite both as correctly-typed primitives (decorator, signature, and one sentence of docstring each — full implementation not required).

Deliverable: 3a's two justifications, 3b's scenario (3-4 sentences), and 3c's corrected sketch.

4. Read the handshake, then read the stateless redesign (1h)

Run `mcp_client_demo.py` and find the `InitializeResult` it prints.

- **4a.** Identify the exact `protocol_version` string printed. Cross-reference `notes/01` §4 and state which spec era this is (classic vs. 2026-07-28) and why the SDK negotiated this version rather than the newest one that exists.
- **4b.** `notes/01` §4 shows a `server/discover` request/response pair from the 2026-07-28 stateless redesign. List three concrete fields present in that JSON that have **no equivalent** in the classic `initialize` exchange this script performs, and for each, say what problem its absence in the classic protocol used to cause (hint: think about what "stateless" removes the need for).
- **4c.** If `minimal_mcp_server.py` were rewritten to speak only the bare 2026-07-28 protocol (no classic handshake), what is the one architectural change to its `CATALOG` global that would matter most, and why (tie this to the "state handle" discussion in `notes/02` §2)?

Deliverable: answers to 4a-4c. 4b must reference actual JSON field names, not paraphrases.

5. Trust-boundary review, applied (1h)

Imagine your team is evaluating three real MCP servers found on the official registry to connect to an internal support agent that already has access to customer PII:

1. A `send_email` server (SMTP relay), verified publisher, minimal permissions requested.
 2. A `web_search` server from an unverified GitHub account with 40,000 recent installs and no visible source code review.
 3. A `filesystem` server scoped, per its own docs, to read/write anywhere on the host machine (no directory allowlist option).
- **5a.** For each, apply `notes/02 §1`'s dependency-vetting framing: what can it *read*, what can it *write/trigger*, and is that consistent with what the agent's task actually needs?
 - **5b.** Which one, combined with the agent's existing PII access, would complete a lethal-trifecta pattern first (untrusted input + sensitive data access + exfiltration channel), and why?
 - **5c.** For the one you'd reject or need to modify before approving, write the two sentences you'd put in a governance intake form (per `notes/02 §4`) explaining the specific risk and the scope reduction you'd require before approval.

Deliverable: a 3-row table (server, read/write summary, verdict) plus 5b's answer and 5c's two sentences.

6. Stretch: MCP vs. the alternatives, applied to a real system (45 min)

For each scenario, name which of {custom tool-calling, OpenAPI wrapped by MCP, plain MCP, A2A} you'd reach for, and defend it in 3-4 sentences using `notes/02 §5`'s comparison table:

1. Your team's own internal Postgres database, used only by one in-house agent your team fully controls.
2. A public weather API you want any Claude user with your server installed to be able to query.
3. Delegating a complex "plan and book a multi-city trip" task to a fully independent travel-booking agent built and operated by a different company, whose internal reasoning you should not need to see.
4. Wrapping an existing, actively-used REST API (that other non-AI services also call) so a model can use it too, without disrupting the existing consumers.

Check: at least one answer should reject MCP even though the scenario involves "an agent using a tool," with a specific reason grounded in the comparison table, not a default.